Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity

Base Paper: Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, Jong C. Park — NAACL 2024

Team Members:
Pranav Moothedath 221AI030
Tarlana Sahil 221AI040

Agenda

- Introduction
- Literature Review
- Outcome of Literature Review
- Problem Statement & Research Objectives
- Existing Methodology (Adaptive-RAG)
- Proposed Enhancements / Novelty
- Dataset Details
- Work Done (Current Status)
- Results & Analysis
- Individual Contributions
- References

Introduction -Background & Motivation

- **Retrieval-Augmented Generation (RAG)** enhances Large Language Models (LLMs) by incorporating external, non-parametric knowledge from knowledge bases (e.g., Wikipedia) to improve factual accuracy in tasks like Question Answering (QA).
- LLMs (GPT-3.5, FLAN-T5, LLaMA) store knowledge in parametric memory, but this knowledge can be incomplete, outdated, or hallucinated.
- **Problem**: Existing RAG approaches use a one-size-fits-all strategy:
- **Single-step RAG** retrieves once → good for simple queries, but fails on complex multi-hop questions.
- **Multi-step RAG** (e.g., IRCoT) iteratively retrieves and reasons → powerful for complex queries, but incurs heavy computational overhead on simple queries.
- **Key Insight**: In real-world scenarios, user queries span a spectrum of complexities — from trivial ("Paris is the capital of what?") to complex multi-hop ("When did the people who captured Malakoff come to the region where Philipsburg is located?").

Introduction - Core Idea of Adaptive-RAG

- **Adaptive-RAG** proposes a novel adaptive QA framework that dynamically selects the most suitable retrieval strategy based on query complexity.
- **Three operating modes:**
- **Level A (No Retrieval):** For straightforward queries answerable by the LLM's own parametric knowledge.
- **Level B (Single-step Retrieval):** For moderate queries requiring one round of document retrieval.
- **Level C (Multi-step Retrieval):** For complex multi-hop queries requiring iterative retrieval and reasoning.
- A lightweight query-complexity classifier (a smaller LM, e.g., T5-Large) is trained to predict the complexity level of each incoming query.
- This classifier is trained on automatically collected labels — no human annotation required.
- **Result:** Adaptive-RAG achieves a strong balance between accuracy (comparable to multi-step methods) and efficiency (significantly faster), as visualized in Figure 1 of the paper.

Literature Review

	Author / Year	Methodology	Advantages	Limitations
1	**Mallen et al. (2023)** — "When Not to Trust Language Models"	Decides whether to retrieve or not based on entity popularity/frequency in the query. Binary decision: retrieve vs. no-retrieve.	Simple and efficient; avoids unnecessary retrieval for well-known entities.	Binary decision is too coarse — cannot handle multi-hop queries that require iterative retrieval. Ignores query complexity beyond entity frequency.
2	**Trivedi et al. (2023)** — IRCoT (Interleaving Retrieval with Chain-of-Thought)	Interleaves Chain-of-Thought (CoT) reasoning with document retrieval iteratively until an answer is derived. Multi-step retrieval-augmented LLM.	Highly effective for complex multi-hop queries. Achieves SOTA on multi-hop QA benchmarks.	Applies the same expensive iterative process to ALL queries regardless of complexity. Significant computational overhead for simple queries.
3	**Asai et al. (2024)** — Self-RAG	Trains the LLM itself to generate special "reflection tokens" that decide when to retrieve, and critiques its own generated output. Single unified model for retrieval + generation.	End-to-end trainable; self-reflective mechanism. Can adaptively decide to retrieve.	Requires training a specialized LLM (not plug-and-play). Uses a different base LM (LLaMA2), making comparisons harder. Suboptimal for handling the full spectrum of complexities.
4	**Qi et al. (2021)** — Answering Open-Domain Questions of Varying Reasoning Steps	Performs a fixed set of operations (retrieve, read, rerank) multiple times until the answer is derived. Built on BERT-like LMs.	Handles varying reasoning steps in a unified framework.	Applies the same fixed operations to every query. Requires additional training of internal LMs. Not designed for modern LLM-based RAG.

Literature Review

	Author / Year	Methodology	Advantages	Limitations
5	**Press et al. (2023)** — Self-Ask	Decomposes multi-hop queries into simpler sub-queries, repeatedly accesses LLMs and retriever to solve sub-queries, and merges solutions.	Effective decomposition for multi-hop reasoning.	Still applies iterative strategy to all queries. Does not pre-assess query complexity.
6	**Jiang et al. (2023)** — FLARE (Active Retrieval Augmented Generation)	Retrieves new documents when generated tokens have low confidence scores. Triggers retrieval adaptively during generation.	Does not blindly retrieve for every query — confidence-based trigger.	Still operates within a single-strategy paradigm. Does not classify queries upfront into complexity levels. May over-retrieve.

Outcome of Literature Review

- **Gap 1:** Existing adaptive retrieval methods (Mallen et al.) make overly simplistic binary decisions (retrieve vs. not), failing to account for multi-hop complexity.
- **Gap 2:** Multi-step approaches (IRCoT, Self-Ask) are powerful but apply the same expensive iterative pipeline to ALL queries, wasting resources on simple ones.
- **Gap 3:** Self-RAG (Asai et al.) requires training a specialized LLM and is not plug-and-play with off-the-shelf LLMs.
- **Gap 4:** No existing work explicitly classifies query complexity into multiple levels (no retrieval, single-step, multi-step) and dynamically routes queries to the appropriate strategy.
- **Core Need:** A framework that can pre-assess query complexity using a lightweight classifier and dynamically route queries to the most suitable retrieval strategy — balancing both accuracy and efficiency.

Problem Statement

Existing retrieval-augmented LLM approaches use a one-size-fits-all strategy that either incurs unnecessary computational overhead on simple queries (multi-step methods) or fails to adequately handle complex multi-hop queries (single-step/no-retrieval methods). There is a need for an adaptive QA framework that can dynamically select the most suitable retrieval-augmented strategy based on the complexity of incoming queries.

Research Objectives

- **Design an adaptive retrieval-augmented generation framework** that can seamlessly switch between no-retrieval, single-step retrieval, and multi-step retrieval strategies based on query complexity.
- **Develop a lightweight query-complexity classifier** (using a smaller LM) that predicts the complexity level of incoming queries without requiring human-annotated training data.
- **Evaluate the framework** on multiple open-domain QA benchmarks (both single-hop and multi-hop) to demonstrate improvements in both accuracy and efficiency over existing adaptive and non-adaptive baselines.

Existing Methodology

Strategy A: No Retrieval

- Formula: $\bar{a} = LLM(q)$
- The LLM generates the answer solely from its parametric knowledge.
- Suitable for straightforward, well-known factual queries.

Strategy B: Single-step Retrieval-Augmented QA

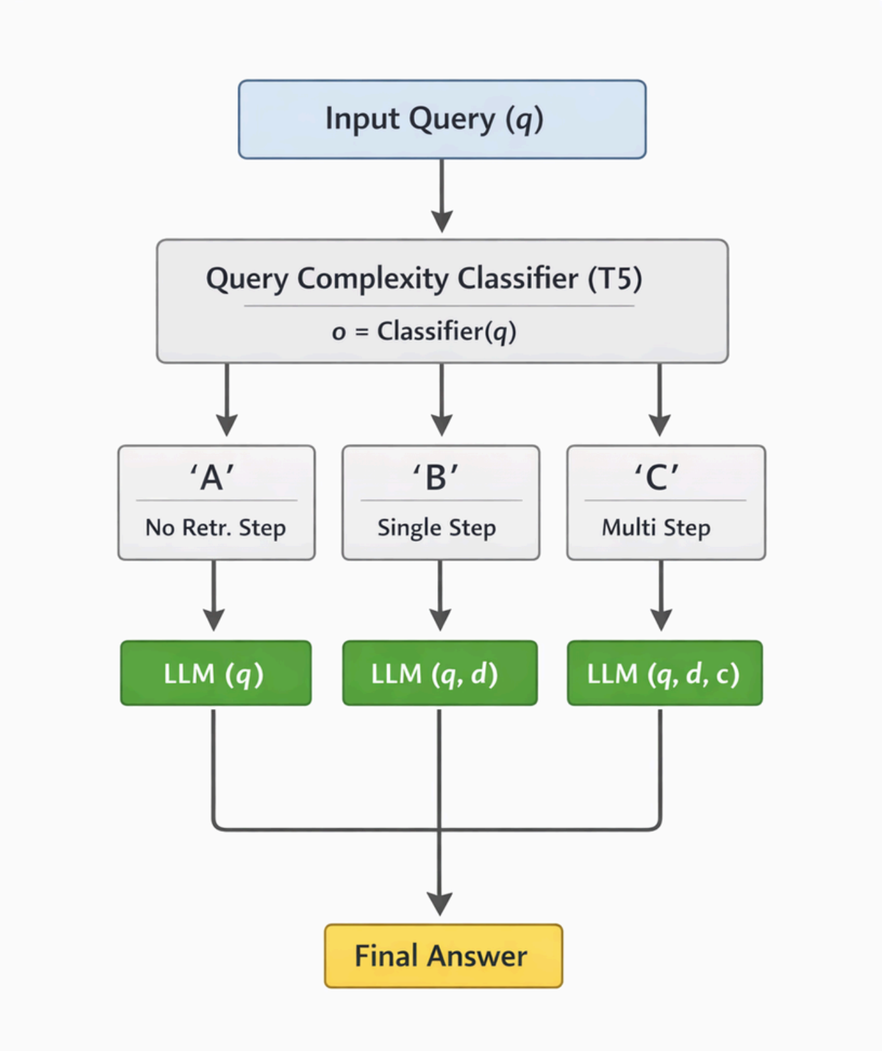
- Formula: $d = Retriever(q; D)$, then $\bar{a} = LLM(q, d)$
- Retrieve relevant documents once using a retriever (e.g., BM25) from an external corpus (e.g., Wikipedia).
- Augment the LLM input with retrieved documents.
- Suitable for moderate queries requiring external knowledge from a single source.

Strategy C: Multi-step Retrieval-Augmented QA

- Formula: At each step i: $d_i = Retriever(q, c_i; D)$, then $\bar{a}_i = LLM(q, d_i, c_i)$
- Iteratively retrieves new documents and generates intermediate answers over multiple rounds.
- Context c_i accumulates previous documents and reasoning outputs.
- Suitable for complex multi-hop queries requiring reasoning over multiple interconnected documents.

Existing Methodology

- The classifier is a **smaller Language Model (T5-Large, 770M params)** that takes the query as input and outputs one of three labels: A (no retrieval), B (single-step), C (multi-step).
- Based on the predicted label, the query is routed to the corresponding retrieval-augmented LLM strategy.
- The LLM and Retriever remain unchanged — no architectural modifications or retraining needed. Only the routing decision changes.



Existing Methodology

Automatic Label Collection (No Human Annotation!)

Step 1: Silver Labels from Model Predictions

- Run all three strategies (No Retrieval, Single-step, Multi-step) on training queries.
- Assign the label of the simplest strategy that correctly answers the query:
- If No Retrieval succeeds → Label A
- If Single-step succeeds (but No Retrieval fails) → Label B
- If only Multi-step succeeds → Label C
- Priority given to simpler models to break ties.

Step 2: Dataset Inductive Bias (for unlabeled queries)

- Queries from single-hop datasets (SQuAD, NQ, TriviaQA) that remain unlabeled → assign B.
- Queries from multi-hop datasets (MuSiQue, HotpotQA, 2Wiki) that remain unlabeled → assign C.

Training:

- Classifier: T5-Large trained with cross-entropy loss.
- Learning rate: 3e-5, optimizer: AdamW.
- Training data: 400 queries per dataset (non-overlapping with test set).
- Best epoch selected within 100 iterations based on validation performance.

Dataset Details

- **Evaluation setting:** All 6 datasets used simultaneously to simulate realistic mixed-complexity query scenarios.
- **Test samples:** 500 per dataset.
- **Retriever:** BM25 (sparse retrieval) used across all methods for fair comparison.
- **External corpus:** Wikipedia (preprocessed by Karpukhin et al. for single-hop; Trivedi et al. for multi-hop).

Dataset	Type	# of Hops	Source	Description
SQuAD v1.1	Single-hop	1	Wikipedia paragraphs	Annotators write questions based on Wikipedia passages
Natural Questions (NQ)	Single-hop	1	Google Search queries	Real user queries from Google Search
TriviaQA	Single-hop	1	Quiz websites	Trivia questions from various quiz sites
MuSiQue	Multi-hop	2–4	Composed single-hop Qs	Single-hop questions composed to form multi-hop queries
HotpotQA	Multi-hop	2	Wikipedia articles	Annotators link multiple Wikipedia articles for questions
2WikiMultiHopQA	Multi-hop	2	Wikipedia + KG paths	Derived from Wikipedia and associated knowledge graph paths

Proposed enhancements

Enhancement Idea 1: Replace BM25 with Dense Retrieval

- The base paper uses BM25 (sparse, term-matching retrieval) throughout.
- Proposal: Replace BM25 with a dense retriever (e.g., Contriever, DPR, or ColBERT) to improve retrieval quality.
- Hypothesis: Semantic retrieval may particularly improve single-step performance, reducing the need for multi-step fallback.

Enhancement Idea 2: Improve Classifier Architecture

- The paper notes a significant gap between the actual classifier and the oracle classifier (e.g., F1: 46.94 vs 56.28 with FLAN-T5-XL).
- The confusion matrix shows 'A' is often misclassified as 'B' (~47%) and 'C' as 'B' (~31%).
- Proposal: Use a more modern classifier backbone (e.g., DeBERTa-v3, or a prompted LLM-based classifier) or incorporate query features like entity count, syntactic complexity, question type.

Enhancement Idea 3: Add a 4th Complexity Level

- Current framework has 3 levels. But intermediate queries (e.g., 2-hop) are lumped with 4-hop queries under 'C'.
- Proposal: Introduce a 4th level (e.g., 'D' for 3+ hop queries) with a dedicated strategy (e.g., tree-based decomposition).

Proposed enhancements

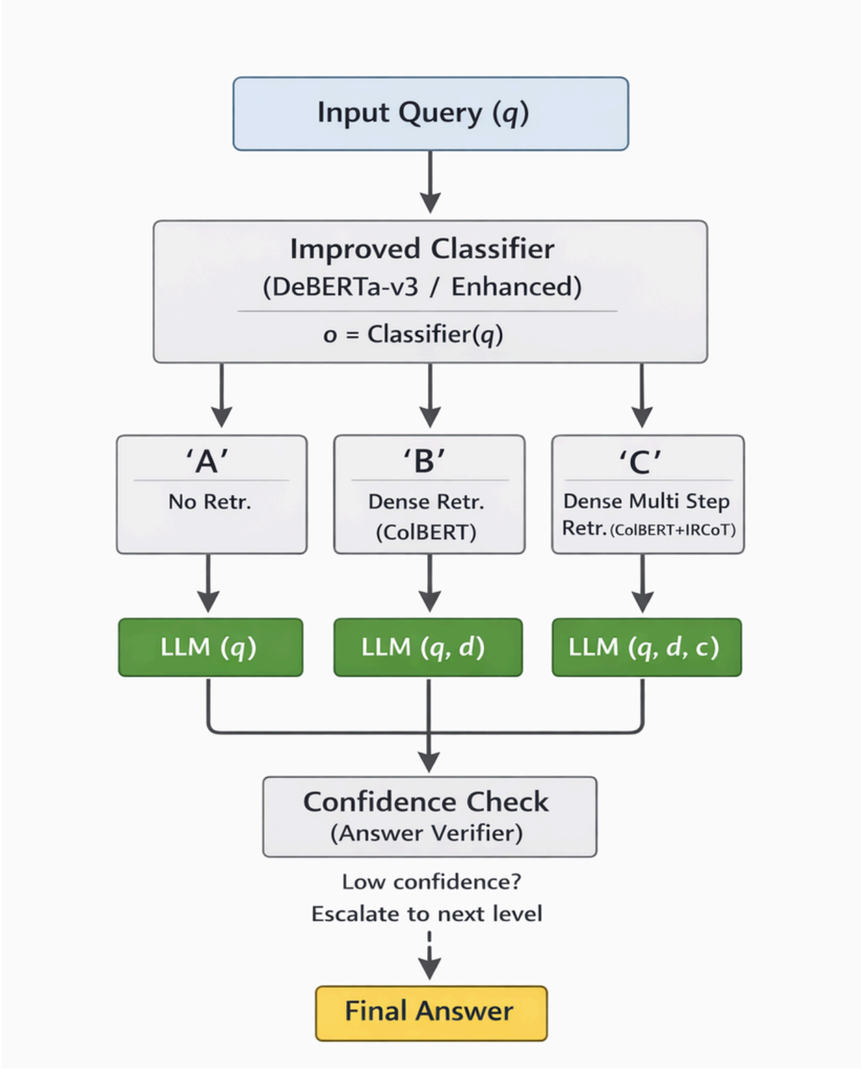
Enhancement Idea 4: Confidence-Based Fallback Mechanism

- Currently, once the classifier predicts a level, there's no fallback if the answer quality is poor.
- Proposal: Add a confidence/verification step — if the generated answer has low confidence, escalate to the next higher complexity strategy.

Enhancement Idea 5: Use Modern LLMs as Base

- The paper uses GPT-3.5-Turbo-Instruct and FLAN-T5 series (2023-era models).
- Proposal: Evaluate with newer open-source LLMs (e.g., LLaMA-3, Mistral, Qwen-2.5) to study how stronger LLMs affect the balance between retrieval strategies.

Proposed enhancements - System Architecture



Changes from base paper are:

- Dense retriever (ColBERT) instead of BM25
- Improved classifier backbone
- Confidence-based fallback/escalation mechanism

Work done

Foundation & Setup Completed

- Completed full reading and understanding of the Adaptive-RAG framework, conducted a literature survey reviewing 6+ related adaptive retrieval papers, and successfully set up the development environment by cloning the Adaptive-RAG repository and configuring all required dependencies.

Current Implementation Phase (Ongoing Work)

- Actively reproducing the base paper results (Table 1) using FLAN-T5-XL and simultaneously integrating dense retrievers such as ColBERT and Contriever into the existing pipeline to enhance retrieval performance.

P**lanned Enhancements & Final Evaluation**

- Upcoming work includes improving the classifier (experimenting with DeBERTa-v3 and additional feature engineering), implementing a confidence-based fallback mechanism, and conducting final evaluation and comparative analysis against the original paper's reported results.

Results and Analysis

Type	Method	EM	F1	Acc	Steps	Relative Time
Simple	No Retrieval	14.87	21.12	15.97	0	0.11
Simple	Single-step	34.83	44.31	38.87	1	1
Adaptive	Adaptive Retrieval	23.87	32.24	26.73	0.5	0.56
Adaptive	Self-RAG*	9.9	20.79	31.57	0.72	0.43
Adaptive	**Adaptive-RAG (Ours)**	**37.17**	**46.94**	**42.1**	**2.17**	**3.6**
Complex	Multi-step	39	48.85	43.7	4.69	8.81
Oracle	Adaptive-RAG w/ Oracle	45	56.28	49.9	1.28	2.11

Key Takeaways:

- Adaptive-RAG achieves ~95-96% of multi-step performance (F1: 46.94 vs 48.85) while being ~2.4x faster (Time: 3.60 vs 8.81).
- Adaptive-RAG outperforms all other adaptive methods (Adaptive Retrieval: 32.24 F1, Self-RAG: 20.79 F1).
- Oracle classifier shows significant headroom (F1: 56.28), indicating that improving the classifier can yield large gains.
- Classifier accuracy distribution: A (No Retrieval): 8.6%, B (Single-step): 53.33%, C (Multi-step): 38.07%.

Results and Analysis

Classifier Accuracy Across Labels

Classifier Size	Overall Acc	No (A)	One (B)	Multi (C)
T5-Small (60M)	53.48%	26.65%	70.62%	53.18%
T5-Base (223M)	53.41%	26.42%	69.46%	56.82%
T5-Large (770M)	54.52%	30.52%	66.28%	65.45%

Efficiency Gain per Label

Label	Time/Query (sec)	% of Queries
A (No Retrieval)	0.35	8.60%
B (Single-step)	3.08	53.33%
C (Multi-step)	27.18	38.07%

Key Observations:

- Classifier size has minimal impact on overall QA performance — even the 60M model works reasonably well (resource-efficient).
- Label 'A' (No Retrieval) is the hardest to classify (~30% accuracy) — often misclassified as 'B'.
- Confusion matrix: 'C' misclassified as 'B' in ~31% of cases; 'A' misclassified as 'B' in ~47% of cases.
- This mis-classification pattern suggests a conservative bias toward single-step retrieval (safer than no retrieval).

Individual Contribution of each student

Sl. No	Name – Reg. No	Details
1	**Sahil – 221AI040**	Led paper reading and complete understanding of the Adaptive-RAG framework, conducted major portion of literature survey, handled environment setup and repository configuration, and currently working on base paper reproduction (FLAN-T5-XL) along with dense retriever integration (ColBERT/Contriever).
2	**Pranav – 221AI030**	Assisted in literature survey and experimental planning, supporting dense retriever integration, and responsible for upcoming classifier improvement experiments (DeBERTa-v3) and implementation of the confidence-based fallback mechanism, followed by final evaluation and comparative analysis.

References

- Jeong, S., Baek, J., Cho, S., Hwang, S.J., & Park, J.C. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. NAACL 2024. arXiv:2403.14403.
- Mallen, A., Asai, A., et al. (2023). When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-parametric Memories. ACL 2023.
- Trivedi, H., Balasubramanian, N., et al. (2023). Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023.
- Asai, A., Wu, Z., et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.
- Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020.
- Robertson, S.E., et al. (1994). Okapi at TREC-3. TREC 1994.
- Rajpurkar, P., et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. EMNLP 2016.
- Yang, Z., et al. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. EMNLP 2018.
- Trivedi, H., et al. (2022). MuSiQue: Multihop Questions via Single-hop Question Composition. TACL.
- Ho, X., et al. (2020). Constructing A Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. COLING 2020.
- Jiang, Z., et al. (2023). Active Retrieval Augmented Generation (FLARE). EMNLP 2023.
- Chung, H.W., et al. (2022). Scaling Instruction-Finetuned Language Models. arXiv:2210.11416.